

Кратко в предыдущих сериях

Советуем перед семинаром освежить в памяти из курса статистики:

- Описательные статистики: среднее, частота абсолютная и относительная, выборочные медиана и квантили, их поиск с помощью пакета `numpy`
- ЦПТ
- Точечные и интервальные оценки
- Оценки параметров среднего и пропорции
- Визуализации с помощью `matplotlib` и `seaborn`
- Работа с датасетами в `pandas`

Тема 2. Как правильно доносить информацию.

0. Цель и задачи

Цель — имея готовые инсайты из исследования, научиться визуализировать и презентовать их так, чтобы в продукт были внесены изменения.

Задачи:

- потренироваться с выбором визуализации
- улучшить визуализацию
- ознакомиться с интерактивными визуализациями с помощью `plotly` и `dash`
- изучить метод донесения информации с помощью фреймворка STAR, опробовать его на практике

1. Вспомним, что было на лекции про визуализации

Вопрос в зал: какие основные принципы хорошей визуализации?

Студенты перечисляют список с лекции

- **Графическая целостность и честность визуализации:** масштабы правильные, оси верны, подписи понятны, не было "подгона" данных
- **Соотношение данных и чернил:** визуализация не перегружена, понятна, из нее легко сделать выводы
- **Многофункциональность графических элементов:** график читается без пояснений, тренды и закономерности легко видны

2. Улучшаем визуализации

2.1 Какая визуализация лучше

На восприятие работы влияет качество визуализации результатов. Плохая структура или неподходящий формат убьют все попытки в качественную аналитику.

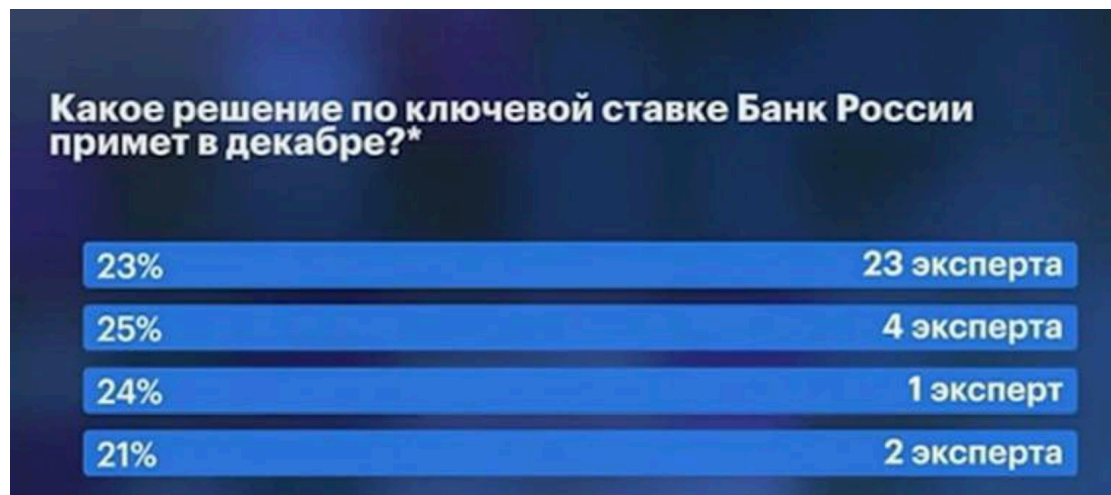
Примеры:

0.1 Без комментариев

 No description has been provided for this image

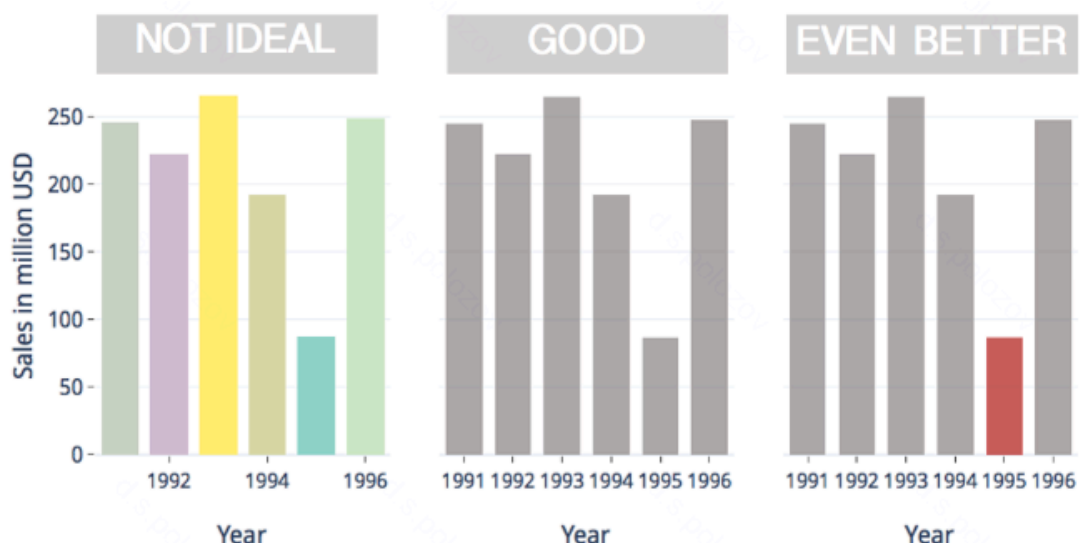
-  *Плохо:* явная "нечестность" визуализации

0.2 Угадайте, по какой логике отсортированы столбцы



1. Дискотека цветов

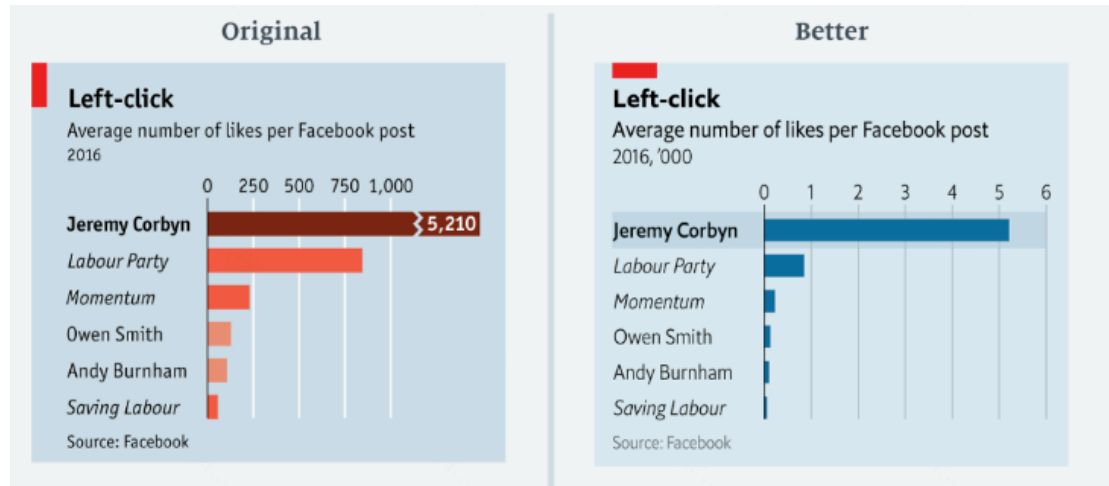
Гистограммы продаж



- ❌ *Плохая*: визуализация перегружена: хаотичные цвета, неполные подписи, сложность сравнения
- ✅ *Хорошая*: единая цветовая гамма, подписи на все года, акцент на важные данные

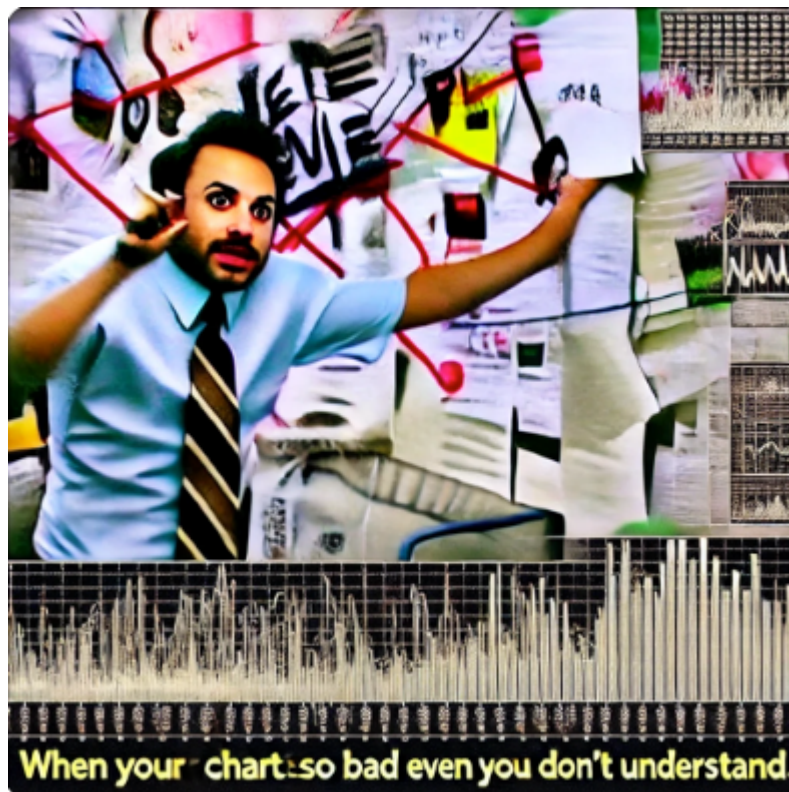
2. Что с масштабом 🤔

Среднее число лайков

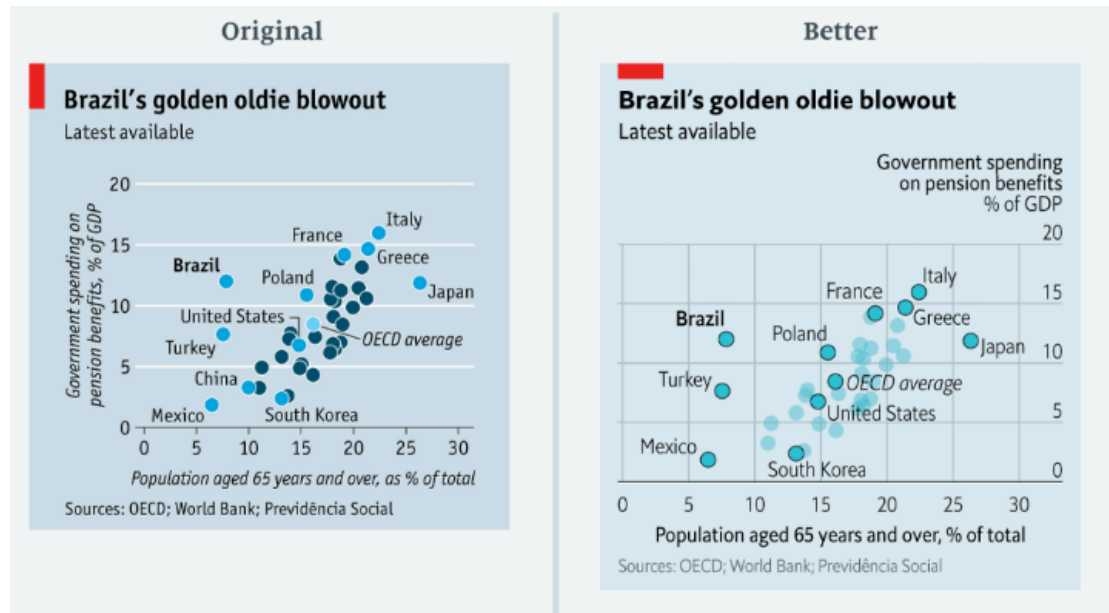


- ❌ *Плохая*: масштаб и абсолютные значения искажают восприятие, непонятно, что означают цвета
- ✅ *Хорошая*: пропорциональный масштаб, удобные числовые обозначения, четкий акцент

3. Зачем столько информации?



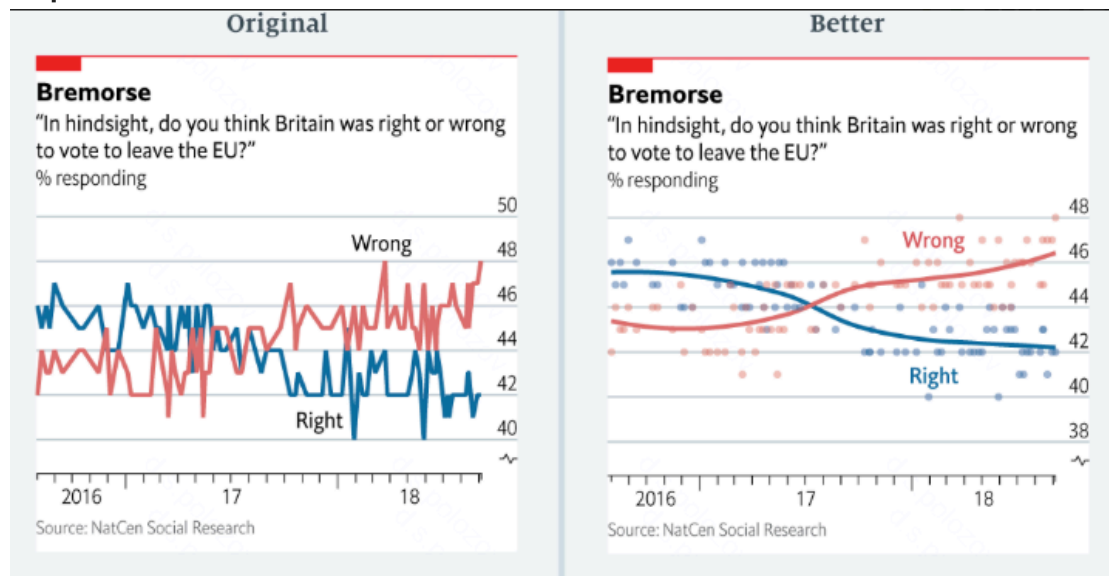
Госрасходы на пенсии



- ✗ *Плохая*: перегруженность, плохо читаемые точки
- ✓ *Хорошая*: оптимизированный размер и цвет точек, логичный масштаб

4. Качаемся на качелях

Опрос по Brexit



- ✗ *Плохая*: резкие линии, мешающие увидеть тренд
- ✓ *Хорошая*: сглаженные тренды, точки данных, четкая цветовая кодировка

Итого:

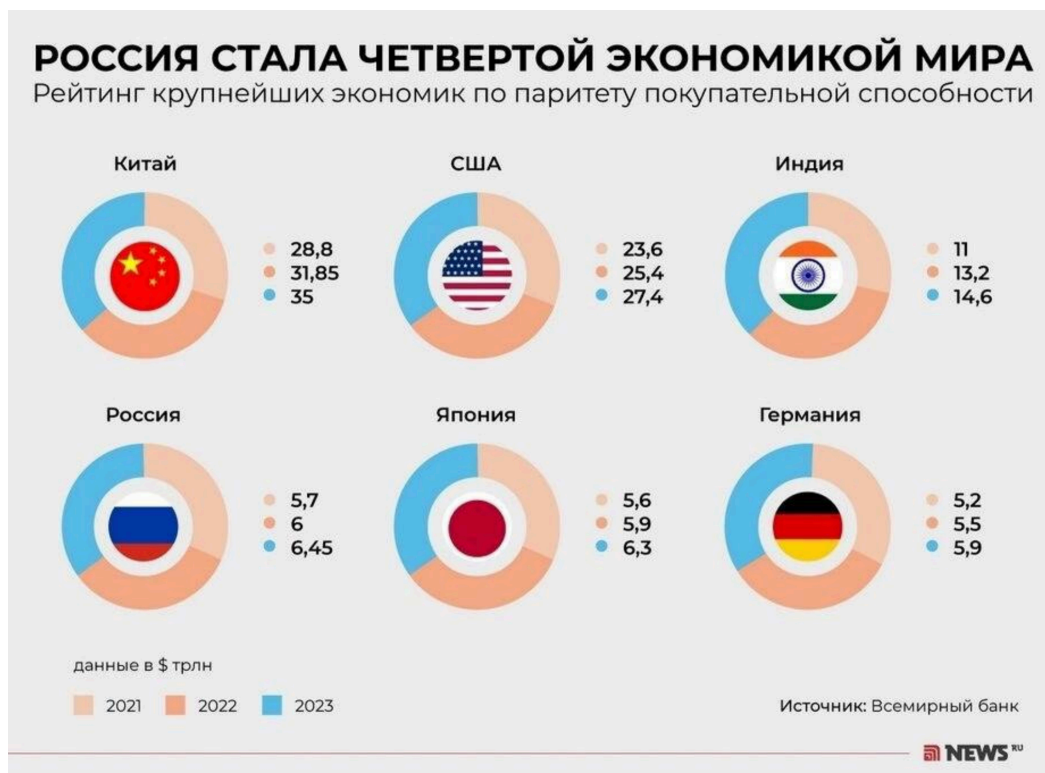
Хорошие визуализации делают данные понятными, убирают лишний шум и подчеркивают ключевые моменты.

- **Плохая визуализация**: график с перегруженной информацией, трудно читаемые цвета, отсутствие подписей.
- **Хорошая визуализация**: четко передает инсайт, легко воспринимается, помогает найти закономерности.

Ключевая мысль: Графики — это инструмент для быстрого понимания данных и донесения мысли. Они помогают находить тренды, закономерности и аномалии.

No description has been provided for this image

2.2 Что не так? Как лучше?





2.2. Кейс на улучшение визуализации

```
In [21]: import matplotlib.pyplot as plt
import scipy.stats as sps
import numpy as np
import seaborn as sns
import pandas as pd
sns.set_theme(style='whitegrid')
```

Загрузка данных

Используем датасет **Titanic**, встроенный в Seaborn.

```
In [2]: # Загружаем данные
df = sns.load_dataset('titanic')
df.head()
```

```
Out[2]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
0	0	3	male	22.0	1	0	7.2500	S	Third	man
1	1	1	female	38.0	1	0	71.2833	C	First	woman
2	1	3	female	26.0	0	0	7.9250	S	Third	woman
3	1	1	female	35.0	1	0	53.1000	S	First	woman
4	0	3	male	35.0	0	0	8.0500	S	Third	man

Описание датасета Titanic

Этот датасет содержит информацию о пассажирах печально известного лайнера *Титаник*, который затонул в 1912 году.

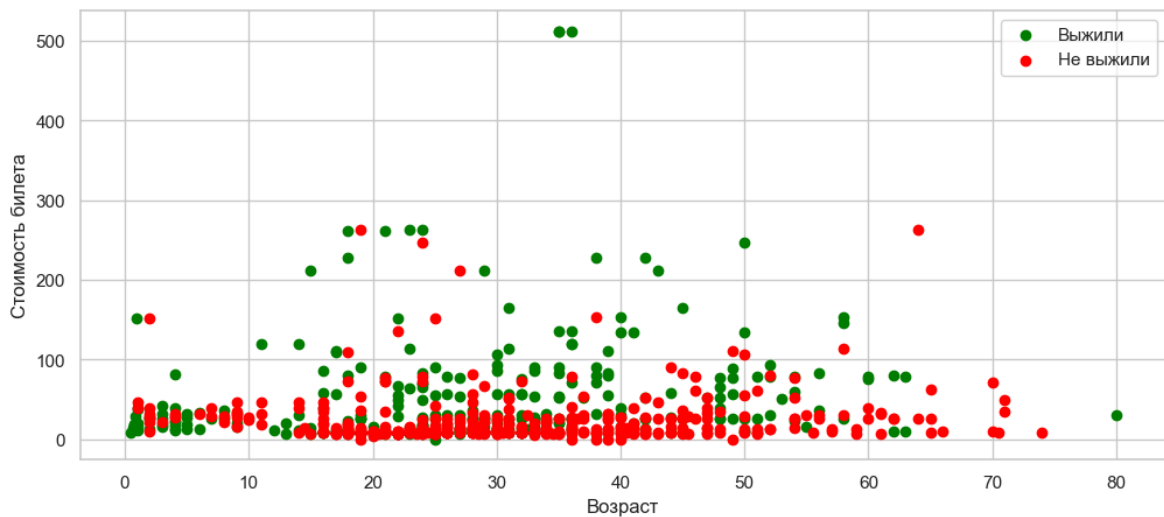
Данные включают характеристики пассажиров, такие как их возраст, пол, класс каюты, а также информацию о выживании.

Описание столбцов:

Колонка	Тип данных	Описание
survived	int (0/1)	Выжил ли пассажир (0 = нет, 1 = да)
pclass	int	Класс каюты пассажира (1 = первый, 2 = второй, 3 = третий)
sex	object	Пол пассажира (male или female)
age	float	Возраст пассажира в годах
sibsp	int	Количество братьев/сестер или супругов на борту
parch	int	Количество родителей/детей на борту
fare	float	Стоимость билета
embarked	object	Порт посадки (C = Cherbourg, Q = Queenstown, S = Southampton)
class	object	Альтернативное обозначение класса каюты (First , Second , Third)
who	object	Категория пассажира (man , woman , child)
adult_male	bool	Является ли пассажир взрослым мужчиной (True/False)
deck	object	Буква палубы каюты (A , B , C и т. д.)
embark_town	object	Название города, где пассажир сел на борт
alive	object	Выжил ли пассажир (yes / no)
alone	bool	Путешествовал ли пассажир в одиночку (True/False)

```
In [3]: df_0 = df[df['survived'] == 0]
df_1 = df[df['survived'] == 1]

plt.figure(figsize=(12, 5))
plt.scatter(x=df_1['age'], y=df_1['fare'], c='green', label='Выжили')
plt.scatter(x=df_0['age'], y=df_0['fare'], c='red', label='Не выжили')
plt.xlabel('Возраст')
plt.ylabel('Стоимость билета')
plt.legend()
plt.show()
```



Дорабатываем график

- Что не так с графиком?
- Доработайте график
- Сделайте вывод о зависимости возраста и стоимости билета

Что не так:

- Слишком много больших и жирных точек, которые накладываются друг на друга
- Непонятна зависимость: кажется, что точки хаотичны
- Можно увеличить масштаб

```
In [4]: df['group_age'] = df['age'] // 5

df['group_age'] = df['group_age'].apply(lambda group: 14. if group > 11 else group)
df['group_age'] *= 5
df['group_age'] += 2.5
df_groupby_age = df.groupby(['group_age', 'survived'])['fare'].median().reset_index()
df_groupby_age.head(5)
```

```
Out[4]:
```

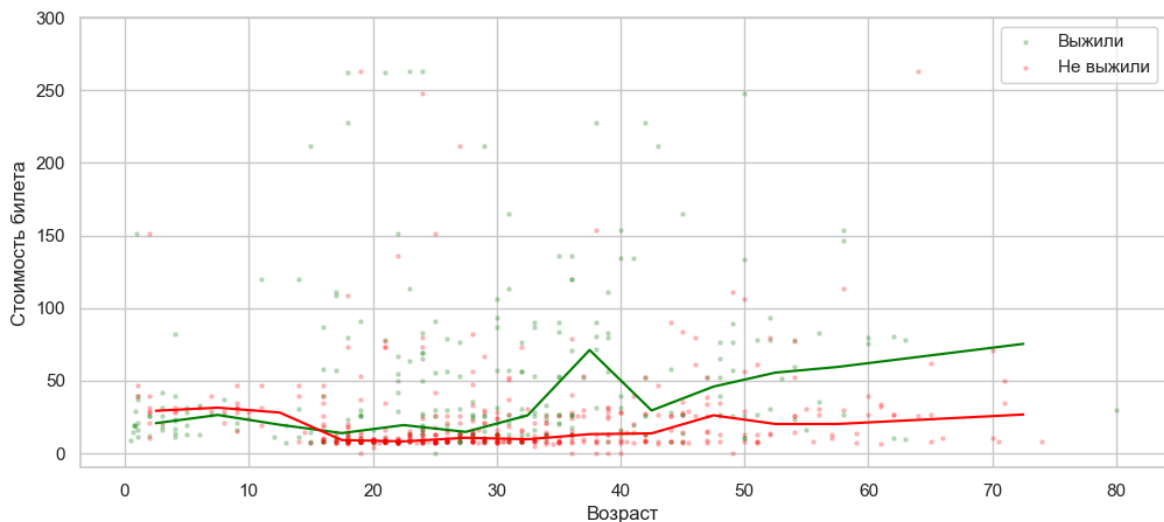
	group_age	survived	fare
0	2.5	0	29.125
1	2.5	1	20.575
2	7.5	0	31.275
3	7.5	1	26.250
4	12.5	0	27.900

Хорошая визуализация

```
In [5]: df_groupby_age_0 = df_groupby_age[df_groupby_age['survived'] == 0]
df_groupby_age_1 = df_groupby_age[df_groupby_age['survived'] == 1]

plt.figure(figsize=(12, 5))
plt.scatter(x=df_1['age'], y=df_1['fare'], c='green', s=5, label='Выжили')
plt.scatter(x=df_0['age'], y=df_0['fare'], c='red', s=5, label='Не выжили')
```

```
plt.plot(df_groupby_age_1['group_age'], df_groupby_age_1['fare'], c='green')
plt.plot(df_groupby_age_0['group_age'], df_groupby_age_0['fare'], c='red')
plt.ylim((-10, 300))
plt.xlabel('Возраст')
plt.ylabel('Стоимость билета')
plt.legend()
plt.show()
```



3. Интерактивные графики в Plotly и Dash

3.1 Основы Plotly: создание интерактивных графиков

```
In [11]: import plotly.io as pio
import plotly.graph_objects as go
import plotly.express as px
pio.renderers.default='notebook'
```

Линейный график

```
In [7]: df_groupby_age['alone'] = df.groupby(['group_age', 'survived'])['alone'].
df_groupby_age['cnt'] = df.groupby(['group_age', 'survived'])['alone'].co
df_groupby_age.head(5)
```

```
Out[7]:
```

	group_age	survived	fare	alone	cnt
0	2.5	0	29.125	0.000000	13
1	2.5	1	20.575	0.000000	27
2	7.5	0	31.275	0.000000	11
3	7.5	1	26.250	0.090909	11
4	12.5	0	27.900	0.222222	9

```
In [8]: # Создание фигуры
fig = go.Figure()

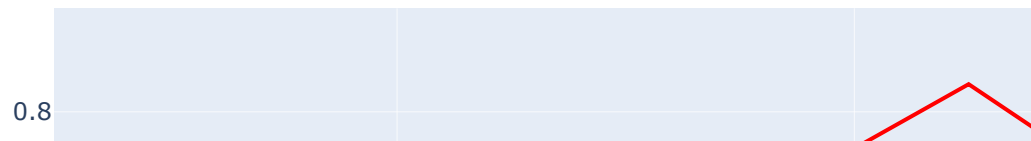
survived = df_groupby_age[df_groupby_age['survived'] == 1]
not_survived = df_groupby_age[df_groupby_age['survived'] == 0]
```

```
fig.add_scatter(x=not_survived['group_age'], y=not_survived['alone'], mode='lines', name='no')
fig.add_scatter(x=survived['group_age'], y=survived['alone'], mode='lines', name='yes')

# Настройка осей и заголовка
fig.update_layout(title="Зависимость доли одиночек от возраста",
                  xaxis_title="возраст", yaxis_title="доля одиночек")

# Отображение
fig.show()
fig.write_html('linear.html')
```

Зависимость доли одиночек от возраста



Улучшим график, добавив доверительные интервалы по ЦПТ

```
In [9]: alpha = 0.95
z = sps.norm.ppf((1 + alpha) / 2)

# Создание фигуры
fig = go.Figure()

survived = df_groupby_age[df_groupby_age['survived'] == 1]
not_survived = df_groupby_age[df_groupby_age['survived'] == 0]

y = not_survived['alone']
std = np.sqrt(y * (1 - y))
n = not_survived['cnt']

fig.add_scatter(x=not_survived['group_age'], y=y, mode='lines', marker_co
```



```

        name='no')
fig.add_traces([go.Scatter(x=not_survived['group_age'], y=np.maximum(0, y
                                mode='lines', line_color = 'rgba(0,0,0,0)',
                                showlegend=False),
                    go.Scatter(x=not_survived['group_age'], y=np.minimum(1, y
                                mode='lines', line_color='rgba(0,0,0,0)', name
                                fill='tonexty', fillcolor = 'rgba(255, 0, 0, 0
                                showlegend=False)])

y = survived['alone']
std = np.sqrt(y * (1 - y))
n = survived['cnt']

fig.add_scatter(x=survived['group_age'], y=y, mode='lines', marker_color=

fig.add_traces([go.Scatter(x=survived['group_age'], y=np.maximum(0, y - z
                                mode='lines', line_color='rgba(0,0,0,0)',
                                showlegend=False),
                    go.Scatter(x=survived['group_age'], y=np.minimum(1, y + z
                                mode='lines', line_color='rgba(0,0,0,0)', name
                                fill='tonexty', fillcolor='rgba(0, 255, 0, 0.2
                                showlegend=False)])

# Настройка осей и заголовка
fig.update_layout(title="Зависимость доли одиночек от возраста",
                    xaxis_title="возраст", yaxis_title="доля одиночек")

# Отображение
fig.show()
fig.write_html('linear_conf_int.html')

```

Зависимость доли одиночек от возраста



Вывод:

- выжившие незначимо чаще были не одни
- ожидаемо несовершеннолетние редко бывали одни на корабле. После 20 лет различия незаметны

Столбчатая диаграмма

Способ 1

```
In [12]: df_groupby_pclass = df.groupby('pclass')['fare'].mean().reset_index()

# Создание столбчатого графика
fig = px.bar(df_groupby_pclass, x='pclass', y='fare', title='Диаграмма це
fig.update_layout(xaxis=dict(tickmode='linear', tick0=1, dtick=1), xaxis_
fig.show()
fig.write_html('bar_1.html')
```

Диаграмма цены от класса



Способ 2

```
In [13]: df_groupby_pclass = df.groupby('pclass')['fare'].mean().reset_index()

# Создание столбчатого графика
fig = px.histogram(df, x='pclass', y='fare', histfunc='avg', title='Диагр

fig.update_layout(xaxis=dict(tickmode='linear', tick0=1, dtick=1), xaxis_
                    bargap=0.2)

fig.show()
fig.write_html('bar_2.html')
```

Диаграмма цены от класса



Практика: постройте гистограмму распределения возраста в зависимости от выживания и сделайте вывод

Способ 1

```
In [14]: # График выживаемости по классам обслуживания
df_groupby_pclass_sex = df.groupby(['pclass', 'sex'])['survived'].mean()

fig_bar = px.bar(df_groupby_pclass_sex, x='pclass', y='survived', color='sex',
                 title='Выживаемость по классам')

fig_bar.update_layout(xaxis_title="класс", yaxis_title="доля выживших")

fig_bar.show()
fig_bar.write_html('bar_divided_1.html')
```

Выживаемость по классам

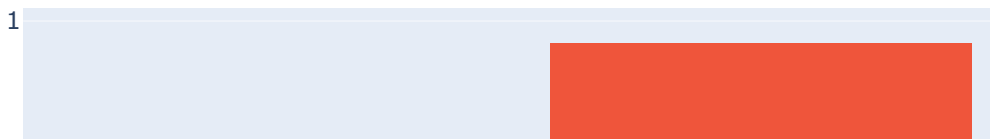


Способ 2

```
In [15]: fig_bar = px.histogram(df, x='pclass', y='survived', color='sex', barmode
          title='Выживаемость по классам')
fig_bar.update_layout(xaxis_title="класс", yaxis_title="доля выживших")

fig_bar.show()
fig_bar.write_html('bar_divided_2.html')
```

Выживаемость по классам



Выводы:

- доля выживших женщин значительно больше, чем доля выживших мужчин. Это подтверждает тезис, что сначала спасали женщин и детей
- вероятность выжить у пассажиров 1 и 2 класса значительно больше, чем у пассажиров классом ниже

Waterfall-графики

Водопадные графики используются чаще всего для анализа итоговой прибыли компании: выявляются главные источники доходов и расходов и находятся сильные и слабые места в экономике продукта

Вернемся к кейсу с Уххххх.Такси

Построим waterfall-график экономики сервиса в месяц. Для этого нам нужны дополнительно данные о расходах:

- аренда авто компанией: 15000₽ в месяц за эконом, 20000₽ в месяц за комфорт и 25000₽ в месяц за комфорт+
- аренда парковки: 5000₽ в месяц за одно парковочное место
- ремонт автомобилей: 5 млн ₽ в месяц суммарно
- бонусные выплаты водителям: 20000₽ единовременно за 250 заказов в месяц

- аренда офиса для IT: 2 млн ₽ в месяц
- зарплата IT-специалистов: 7 млн ₽ в месяц суммарно
- затраты на рекламу: 7 млн ₽ в месяц суммарно
- прочие затраты: 5 млн ₽ в месяц суммарно

Посчитаем нужные нам величины и отобразим их на графике

```
In [25]: df = pd.read_csv('orders_group.csv')
df
```

```
Out[25]:
```

	taxist_id	date	orders_cnt	total_cost	total_service_fee	auto_id	tariff
0	1	2024-09-01	12	8674	1734.8	1494	эконом
1	1	2024-09-02	10	8136	1627.2	1494	эконом
2	1	2024-09-03	9	6361	1272.2	1494	эконом
3	1	2024-09-04	13	9576	1915.2	1494	эконом
4	1	2024-09-05	7	3664	732.8	1494	эконом
...	...	...	...	...	...	...	...
32909	1609	2024-09-19	14	21130	4226.0	565	комфорт
32910	1609	2024-09-22	8	10786	2157.2	565	комфорт
32911	1609	2024-09-24	13	15341	3068.2	565	комфорт
32912	1609	2024-09-25	8	6322	1264.4	565	комфорт
32913	1609	2024-09-30	14	15372	3074.4	565	комфорт

32914 rows x 10 columns

```
In [48]: # Доходы сервиса
df_group = df.groupby('tariff')[
    'orders_cnt', 'total_cost', 'total_service_fee',
    'rent_cost', 'driver_income', 'company_income'
].sum().reset_index()
df_group
total_fee_income = df_group['total_service_fee'].sum()
```



```
total_rent_income = df_group['rent_cost'].sum()
total_fee_income, total_rent_income
```

Out [48]: (56265065.0, 61822450)

In [55]: *# Расходы сервиса на авто*

```
c_auto_econom = 15_000
c_auto_comfort = 20_000
c_auto_comfort_plus = 25_000
c_parking_per_one = 5_000
c_repair = 5_000_000

cars = pd.read_csv('cars_clean.csv')
cars['rent_company_cost'] = cars['tariff'].apply(
    lambda t: c_auto_econom if t == 'эконом' else \
    c_auto_comfort if t == 'комфорт' else c_auto_comfort_plus)
cars['parking_cost'] = c_parking_per_one
cars_costs = cars.groupby('tariff')[['rent_company_cost', 'parking_cost']]
total_rent_company_cost = -cars_costs['rent_company_cost'].sum()
total_parking_cost = -cars_costs['parking_cost'].sum()
total_repair_cost = -c_repair
total_rent_company_cost, total_parking_cost, total_repair_cost
```

Out [55]: (-32815000, -9065000, -5000000)

In [56]: *# Расходы сервиса на бонусы водителям*

```
cnt_orders_bonus = 250
bonus_amt = 20_000

df_driver = df.groupby('taxist_id')[['orders_cnt']].sum()
df_driver['bonus'] = df_driver['orders_cnt'].apply(
    lambda cnt: 0 if cnt < cnt_orders_bonus else bonus_amt
)
total_drivers_bonuses_costs = -df_driver['bonus'].sum()
total_drivers_bonuses_costs
```

Out [56]: -3620000

In [57]: *# Остальные расходы сервиса*

```
total_office_cost = -2_000_000
total_it_salary_cost = -7_000_000
total_ad_costs = -7_000_000
total_another_costs = -5_000_000
```

Построим waterfall-график

In [68]: *# Исходные значения*

```
values = [
    total_fee_income,
    total_rent_income,
    total_rent_company_cost,
    total_parking_cost,
    total_repair_cost,
    total_drivers_bonuses_costs,
    total_office_cost,
```

```

total_it_salary_cost,
total_ad_costs,
total_another_costs,
0 # total – Plotly считает сам
]
values = np.round(values, -5)

fig = go.Figure(go.Waterfall(
    name = "2024",
    orientation = "v",
    measure = [
        "relative", # доход комиссии
        "relative", # доход аренда авто
        "relative", # расходы аренда авто
        "relative", # расходы парковка
        "relative", # расходы ремонт
        "relative", # расходы бонусы водителям
        "relative", # расходы офис
        "relative", # расходы IT зарплата
        "relative", # расходы реклама
        "relative", # расходы прочие
        "total"      # итог
    ],
    x = [
        "Доход: комиссия",
        "Доход: аренда авто",
        "Расход: аренда авто",
        "Расход: парковка",
        "Расход: ремонт авто",
        "Расход: бонусы водителям",
        "Расход: офис",
        "Расход: IT зарплата",
        "Расход: реклама",
        "Расход: прочие",
        "Итоговая прибыль"
    ],
    textposition = "outside",
    y = values,
    # text = text_labels, # ← добавляем подписи
    connector = {"line":{"color":"rgb(63, 63, 63)"}}},
    increasing = {"marker":{"color":"green"}}},
    decreasing = {"marker":{"color":"red"}}},
    totals = {"marker":{"color":"blue"}}},
))

fig.update_layout(
    title = "Waterfall: экономика сервиса за месяц",
    waterfallgap = 0.3,
)

fig.show()
fig.write_html("economics.html") # Сохраняем в HTML

```

Waterfall: экономика сервиса за месяц



Heatmap-графики

Heatmap используются для анализа совместного распределения двух признаков. С помощью тепловых карт можно выявить корреляцию между признаками, оценить совместное распределение и разбить данные на сегменты. Обычно heatmap строят, когда хотят найти зависимости ключевых метрик друг от друга либо для первичного анализа данных

```
In [69]: orders = pd.read_csv('orders_clean.csv')
orders
```

Out [69]:

	id	data	taxist	client	r1	r2	tariff	len	time	cost
0	1	2024-09-16	151.0	1.0	NaN	5.0	эконом	10.0000	19.00	1.0
1	2	2024-09-08	511.0	1.0	NaN	5.0	эконом	11.7000	22.00	0.9
2	3	2024-09-14	632.0	1.0	NaN	5.0	эконом	14.1600	18.14	1.2
3	4	2024-09-03	1439.0	2.0	NaN	5.0	комфорт	27.6000	45.00	1.3
4	5	2024-09-19	292.0	2.0	5.0	NaN	комфорт	44.6000	72.33	0.9
...	...	...	...	...	...	...	...	...	...	...
341304	341305	2024-09-21	1288.0	97566.0	NaN	5.0	комфорт	37.8600	65.30	1.2
341305	341306	2024-09-17	1421.0	97567.0	5.0	5.0	эконом	59.8000	84.50	1.1
341306	341307	2024-09-09	680.0	97567.0	NaN	5.0	эконом	20.2400	40.20	1.2
341307	341308	2024-09-01	1039.0	97567.0	NaN	5.0	эконом	6.0000	9.00	1.3
341308	341309	2024-09-05	584.0	97567.0	5.0	NaN	эконом	26.1178	34.40	0.8

341309 rows x 13 columns

```

In [83]: orders_small = orders[orders['cost'] < 1_500]

fig = px.density_heatmap(
    orders_small,
    x="len",
    y="cost",
    nbinsx=30,
    nbinsy=30,
    marginal_x="histogram", # маргинальный график по X
    marginal_y="histogram", # маргинальный график по Y
)

fig.update_layout(
    title="Heatmap с маргинальными распределениями: Длина поездки vs Стои
)

fig.show()

```

Heatmap с маргинальными распределениями: Длина



3.2 Введение в Dash

Интерактивный график с карт

!pip install dash

```
In [84]: # Загружаем датасет с данными цен квартир (запусти, если выполняешь вперв
df = pd.read_csv('input_data.csv', sep=';', decimal=',')
df['geo_lat'] = df['geo_lat'].astype('float')
df['geo_lon'] = df['geo_lon'].astype('float')

# Загружаем датасет с регионами
regions = pd.read_excel('regions.xlsx')
regions['id_region'] = regions['GNINMB,C,4'] // 100
regions['name_region'] = regions['NAME,C,40']
```

```
In [88]: import dash
from dash import dcc, html, Input, Output

# Функция для агрегации данных по количеству комнат
def aggregate_data(rooms=None):
    filtered_df = df if rooms is None else df[df['rooms'] == rooms]
    metro_summary = filtered_df.groupby("id_region").agg(
        count=("price", "count"),
        median_price=("price", "median"),
        lat=("geo_lat", "median"),
```

```

        lon=("geo_lon", "median")
    ).reset_index()
    metro_summary['cost'] = metro_summary['median_price'].apply(lambda pr
    return metro_summary.merge(regions[['id_region', 'name_region']], on=

# Создаем дашборд
app = dash.Dash(__name__)
app.layout = html.Div([
    html.H1("Карта цен на квартиры в России по регионам"),
    html.Label("Выберите количество комнат:"),
    dcc.Dropdown(
        id='room-filter',
        options=[
            {'label': 'Все', 'value': 'all'}
        ] + [{'label': f'{i} комн.', 'value': i} for i in sorted(df['room
        value='all',
        clearable=False
    ),
    dcc.Graph(id="russia_map")
])

@app.callback(
    Output("russia_map", "figure"),
    Input("room-filter", "value")
)
def update_map(selected_rooms):
    rooms = None if selected_rooms == 'all' else int(selected_rooms)
    metro_summary_region = aggregate_data(rooms)
    fig = px.scatter_mapbox(
        metro_summary_region,
        lat="lat",
        lon="lon",
        size="count",
        color="median_price",
        hover_name="name_region",
        title="Стоимость квартир и количество предложений по регионам Рос
        color_continuous_scale="Bluered",
        size_max=20,
        center={"lat": 55, "lon": 55}, # Центрируем карту
        zoom=3,
        mapbox_style="open-street-map"
    )
    fig.write_html("map.html") # Сохраняем в HTML – там можно будет посмо
    return fig

app.run_server(debug=True, port=8051)

```

Карта цен на квартиры в России по регионам

Выберите количество комнат:

Все



Стоимость квартир и количество предложений по ре



4. Фреймворк STAR

Вопрос в зал: что такое фреймворк STAR?

Фреймворк STAR (Situation, Task, Action, Result) — это метод структурированного представления информации, который помогает четко и логично излагать идеи. В контексте визуализации данных он может использоваться для структурирования анализа, презентации выводов и объяснения методологии.

Пример использования: я поискал на habr'е информацию про фреймворк STAR и обнаружил в топе страниц следующие:

- [Как подготовиться к собеседованию в зарубежных компаниях по методике STAR и почему это не очередная выдумка HR](#)

- Как я перестал превращать собес в экзамен: оцениваем хард- и софт-скиллы за одно собеседование

Как видите, этот фреймворк довольно популярен в подготовке и проведении собеседования

Формат STAR:

1. **Situation (Ситуация):** Контекст проблемы или задачи.
 2. **Task (Задача):** Конкретная цель, которую нужно достичь.
 3. **Action (Действие):** Шаги, предпринятые для решения задачи.
 4. **Result (Результат):** Итог выполненных действий.
-

Кейс: Борьба с зависанием в соцсетях



- **S (Ситуация):** Я заметил, что слишком много времени провожу в соцсетях, из-за чего не успеваю делать важные дела. (рабочие задачи, учёба, домашние дела, хобби, etc)
 - **T (Задача):** Я поставил перед собой задачу понять, сколько времени уходит на соцсети и как это влияет на продуктивность.
 - **A (Действие):** Я начал отслеживать время использования телефона. Понял, что в дни зависания в соцсетях у меня не хватало времени на другие активности, и я просрачивал дедлайны или у меня не было времени на мои хобби.
 - **R (Результат):** После проведенного анализа я решил поставить лимиты на использование некоторых приложений. В итоге свободного времени стало больше, а дела делаются быстрее. Правда, есть один минус: теперь SLA моих ответов стал немного больше. 😊
-



Артём Трещенко
@treschenko24

Читать

Когда все вокруг говорят про финансовую грамотность, а ты просто чилловый парень, который тратит деньги так, как чувствует



- **S (Situation) — Ситуация:** Около года назад я понял, что достаточно много трачу: несмотря на солидную зарплату накоплений практически не было.
- **T (Task) — Задача:** Я поставил перед собой цель — научиться лучше управлять своими деньгами, создать подушку безопасности и начать инвестировать, чтобы накопить на квартиру.
- **A (Action) — Действия:** Я начал с малого: завёл таблицу расходов и стал фиксировать каждую трату. Затем прошёл онлайн-курс по инвестициям, где научился разбираться в базовых инвестиционных инструментах. Также я открыл брокерский счёт и начал инвестировать небольшие суммы в индексные фонды.
- **R (Result) — Результат:** Сейчас у меня есть подушка безопасности на примерно год жизни. Не знаю, повлияла ли моя работа над финансовой грамотностью или повышение зп, но теперь у меня получается откладывать. Хотелось бы сказать, что мои инвестиции принесли первые доходы, но с текущей ситуацией это сложно, так что рассматриваю инвестиции как первый опыт, на котором пытаюсь набить шишки 🤔

5. 📌 Практика: презентация результатов по STAR

Студенты разбиваются на 4 группы. Каждая группа получает ситуацию и визуализацию. Каждая группа должна додумать, что они сделали в кейсе, что получили и что предлагают изменить в продукте

Подготовка студентов и короткое выступление по STAR

Обратная связь от участников и семинариста

Кейс 1. Проседание покупок

Ситуация: В мобильном приложении для заказа еды изменили экран онбординга — теперь регистрация обязательна перед показом каталога. После обновления аналитик проверяет воронку покупок: из-за обязательной регистрации часть пользователей ушла с воронки, однако мы получили прирост в регистрациях, то есть, больше информации о новых пользователях

Задача студентов: Использовать фреймворк **STAR**, чтобы разобрать:

- S: Почему изменился онбординг
- T: Что хотим понять (влияние на конверсию)
- A: Что делаем (анализ воронки)
- R: Что решаем (гипотезы про упрощение регистрации)

График:

```
In [102... data = dict(Quantity=[1, 0.25, 0.2, 0.12,
                      1, 0.5, 0.16, 0.10],
            Stage=['Заходы', 'Регистрации', 'Добавления в корзину', 'Опла
            Process=['Старый']*4 + ['Новый']*4)

fig = px.funnel(data, y='Stage', x='Quantity', color='Process',
               color_discrete_map={"Старый": "#374B53",
                                   "Новый": "#A4B7C8"},
               template="simple_white",
               title='Добавление обязательной регистрации',
               labels={"Stage": ""})

fig.show()
```

Добавление обязательной регистрации

Заходы



Кейс 2. Резкий рост оттока

Ситуация: Подписной сервис (онлайн-курсы) запустил акцию: первый месяц подписки с большой скидкой. Retention пользователей новой акции оказался хуже, однако мы привлекли новых пользователей. Что делать?

Задача студентов: Использовать **STAR**, чтобы разобраться:

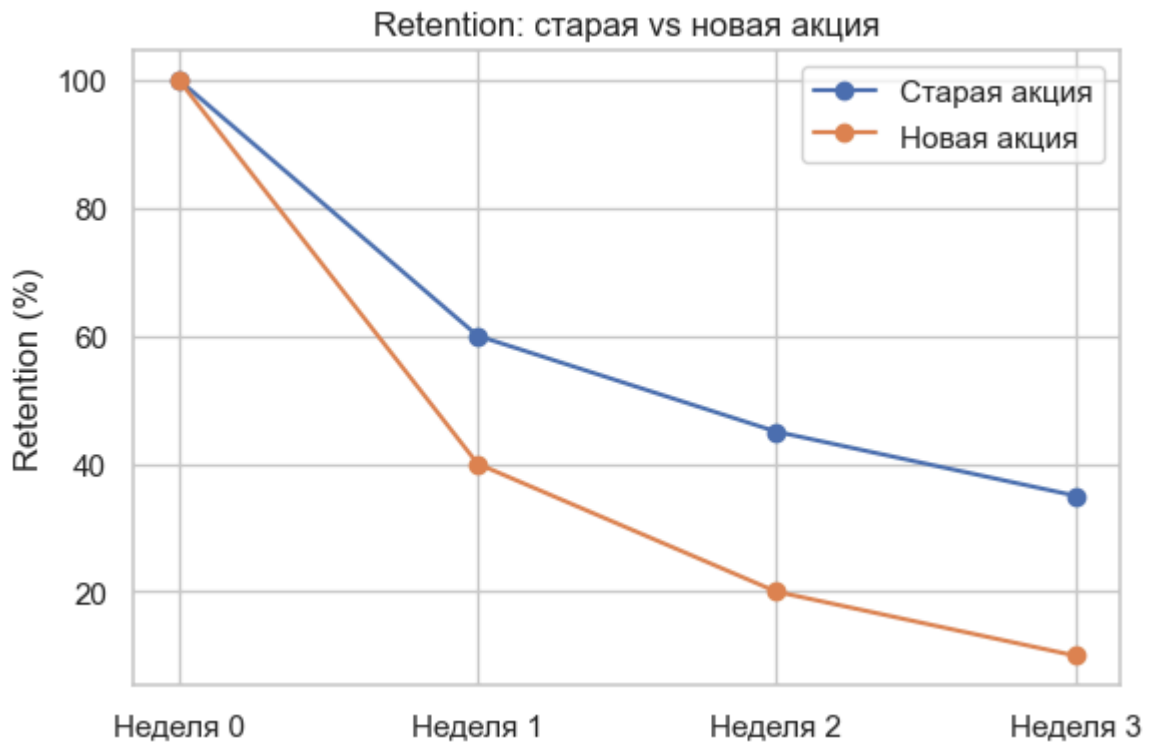
- S: Акция → много новых пользователей
- T: Проверить поведение (retention, LTV)
- A: Сравнить когортные графики
- R: Принять решение об изменении условий или сегментировании

График:

```
In [103... weeks = ['Неделя 0', 'Неделя 1', 'Неделя 2', 'Неделя 3']
old = [100, 60, 45, 35]
new = [100, 40, 20, 10]

plt.figure(figsize=(6, 4))
plt.plot(weeks, old, marker='o', label='Старая акция')
plt.plot(weeks, new, marker='o', label='Новая акция')
plt.title('Retention: старая vs новая акция')
plt.ylabel('Retention (%)')
```

```
plt.legend()
plt.tight_layout()
```



Кейс 3. Локальное падение NPS

Ситуация: Сервис доставки продуктов отслеживает NPS по городам. В одном городе NPS резко упал за неделю. Что могло пойти не так?

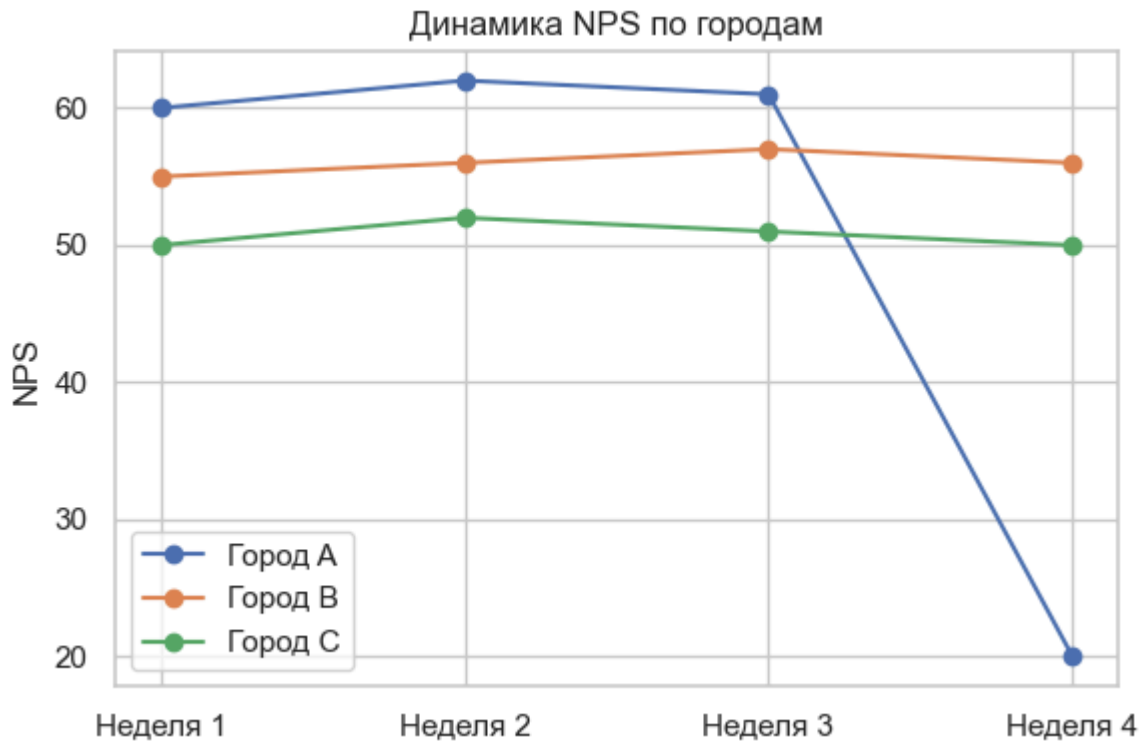
Задача студентов: Использовать **STAR**, чтобы разобраться:

- S: Локальное падение
- T: Найти причину
- A: Проверить жалобы, сроки доставки, партнёров
- R: Выработать решение (замена партнёра, компенсации)

График:

```
In [104... weeks = ['Неделя 1', 'Неделя 2', 'Неделя 3', 'Неделя 4']
city_a = [60, 62, 61, 20]
city_b = [55, 56, 57, 56]
city_c = [50, 52, 51, 50]

plt.figure(figsize=(6, 4))
plt.hist(weeks, city_a, marker='o', label='Город А')
plt.plot(weeks, city_b, marker='o', label='Город В')
plt.plot(weeks, city_c, marker='o', label='Город С')
plt.title('Динамика NPS по городам')
plt.ylabel('NPS')
plt.legend()
plt.tight_layout()
```



Кейс 4. Тест нового сценария оплаты

Ситуация: Интернет-магазин протестировал новую кнопку «Оплатить позже». Ожидали рост конверсии, но она упала.

Задача студентов: Использовать **STAR**, чтобы разобрать:

- S: Новый сценарий оплаты
- T: Проверить эффект
- A: A/B тест → анализ разницы
- R: Принять решение — откатить или улучшить сценарий

График:

```
In [98]: data = dict(Quantity=[1, 0.2, 0.15,
                             1, 0.23, 0.12],
                    Stage=['Сессии', 'Добавления в корзину', 'Оплаты']*2,
                    Process=['Старый']*3 + ['Новый']*3)

fig = px.funnel(data, y='Stage', x='Quantity', color='Process',
                color_discrete_map={'Старый': "#374B53",
                                    "Новый": "#A4B7C8"},
                template="simple_white",
                title='Новый сценарий оплаты',
                labels={"Stage": ""})

fig.show()
```

Новый сценарий оплаты



6. Итоги семинара

С чем выходим с занятия:

- повторили, как нужно и как не нужно графически изображать данные
- научились интерактивным визуализациям с помощью `plotly` и `dash`
- изучили фреймворк STAR
- потренировались презентовать результаты перед заказчиком

7. Полезные материалы

- [Статья с плохими визуализациями](#)
- [Еще одна статья про плохие визуализации](#)
- [ТГ-канал "отвратительные графики"](#)